

Conversor e Validador de Dados Climáticos de Manaus

Projeto Prático de Programação em Python

Dupla: Jéssica Lange Miranda Barbosa e Sadrack Maia Jesus

Curso: Técnico em Inteligência Artificial

Componente Curricular: Programação em Python

GitHub: https://github.com/langik-crypto/Projeto_Python_Tec._IA_SaJe/tree/main



| CONTEXTUALIZAÇÃO

Importância da Análise

O monitoramento climático em regiões como Manaus gera um grande volume de dados essenciais para o estudo de variações térmicas e condições ambientais.

O Problema dos Dados Brutos

Registros meteorológicos frequentemente chegam com falhas de formatação ou dados corrompidos (ex: "32.5C,85%" ou strings inválidas como "abc").

A Solução Desenvolvida

Um programa automatizado em Python focado na limpeza, validação prévia e classificação correta de cada entrada.



Simulação Real

Tratamento automatizado simulando os fluxos reais de uma estação meteorológica amazônica.

OBJETIVOS DO PROJETO

Geral

Processar e validar registros climáticos simulados de forma eficiente e robusta, eliminando ruídos operacionais.

Integridade

Validar a integridade estrutural e os limites numéricos aceitáveis de cada dado de entrada.

Transformação

Converter com precisão escalas térmicas (Celsius para Fahrenheit) e gerar relatórios quantitativos.

Classificação Lógica: O sistema aplica inteligência baseada em regras para definir a sensação térmica final.

| FUNCIONAMENTO DO PROGRAMA



1. Entrada

Lista contendo strings brutas da estação ("TempC, Umidade%").



2. Parsing

Isolamento dos dados usando métodos nativos como `.split(",")`.



3. Validação

Verificação de tipos numéricos e limites plausíveis.



4. Processo

Cálculo da conversão de escala e aplicação de regras de decisão.



5. Relatório

Métricas e sumário geral com logs consolidados do dia.

| ESTRUTURA DO CÓDIGO

Modularidade e Defesa

Funções Reutilizáveis:

`celsius_para_fahrenheit(c)` e `sensacao(temp, umid)` isolam as regras de negócio.

Programação Defensiva:

Uso estratégico de blocos `try-except` captura dados espúrios ou nulos sem interromper o fluxo do script.

Estruturas de Controle:

Laços `for` iteram sobre os lotes, guiados por decisões condicionais `if-elif-else`.

```
def celsius_para_fahrenheit(c): return (c *  
9/5) + 32  
def sensacao(temp, umidade):  
    if temp >= 35 and umidade >= 80: return  
        "Extremamente Quente"  
    elif temp >= 30: return "Muito Quente"  
    return "Agradável"
```

EXEMPLO PRÁTICO DE EXECUÇÃO

Abaixo está o rastreamento completo de como o algoritmo processa uma entrada típica de Manaus:

Fase do Processo	Operação Interna	Resultado / Estado
Dado de Entrada	Leitura da string bruta em lote	"32.5C, 85%"
Conversão Térmica	Aplicação da fórmula matemática direta	90.5 °F
Regra de Sensação	Validação lógica cruzando Temperatura e Umidade	Muito Quente
Saída do Log	Geração da string estruturada final	Registro Válido Concluído



| RESULTADOS ALCANÇADOS

Filtragem Inteligente

Identificação instantânea de anomalias (como "50C, 30%" fora dos limites ou strings corrompidas "abc").

Processamento em Lote

Automação completa eliminando a necessidade de revisão manual de planilhas de monitoramento.

Confiabilidade

Garantia de que apenas dados consistentes alimentarão os modelos subsequentes de análise ou IA.

| POSSÍVEIS MELHORIAS

Persistência e Arquivos

Implementar suporte nativo para leitura e escrita direta em arquivos estruturados como CSV, JSON ou bancos de dados SQL.

Interface e Dashboards

Criação de uma interface gráfica amigável (GUI) e conexão com Power BI para relatórios visuais dinâmicos.

Escalabilidade e APIs

Conexão direta com APIs meteorológicas reais (ex: INMET / OpenWeatherMap) para captura em tempo real.

Modelos de Inteligência

Aplicação prática de algoritmos de Machine Learning para modelagem preditiva baseada no histórico limpo.

| CONCLUSÃO

O projeto sintetiza os fundamentos de **lógica, estruturas de controle e tratamento de erros** em Python. Demonstra na prática o passo inicial mais crítico de qualquer solução em Inteligência Artificial: **o pré-processamento e higienização de dados brutos**.

- Manipulação precisa de strings
- Robustez com tratamento de exceções
- Funções modulares e limpas
- Arquitetura de código defensiva

REFERÊNCIAS E FERRAMENTAS

Para o desenvolvimento deste projeto, utilizamos as seguintes ferramentas:



Python



Google Colab



Gemini



GitHub

Apresentação elaborada com o auxílio do Gemini (Google AI),
com adaptação e revisão dos autores

OBRIGADO!

Perguntas ou Considerações?

Dupla: Jéssica Lange & Sadrack Maia
Técnico em Inteligência Artificial

